

Fusing the Transformer-FFN Epilogue cuBLAS Cannot Fuse

A whole-program-fusion structural advantage for $\text{GeLU}(AB + \text{bias})$: one launch and one HBM write versus three

hexa-lang GPU codegen substrate

dancinlab / hexa-lang

github.com/dancinlab/hexa-lang

2026-05-25

Abstract

A cuBLAS-using neural-network stack cannot fuse a bias-add or an activation into the epilogue of its general matrix multiply (plain `cublasSgemm/cublasGemmEx` expose no such epilogue), so the transformer feed-forward block $C = \text{GeLU}(AB + \text{bias})$ is forced to round-trip the $M \times N$ result through high-bandwidth memory (HBM): one kernel writes the GEMM output, a second reads and rewrites it to add the bias, a third reads and rewrites it to apply GeLU — three kernel launches and three full-tensor HBM write passes. We emit a single hexa-native NVPTX kernel that fuses the GEMM, the bias-add, and the GeLU into one epilogue and writes the $M \times N$ result to HBM exactly once. We pre-register a falsifier (F-FUSION-EPILOGUE-GEMM-BIAS-GELU) and settle it with a \$0 deterministic structural oracle that counts launches and HBM write-bytes by construction: at the LLaMA-7B feed-forward shape ($M = 4096$, $N = 11008$, $K = 4096$; C tensor = 180 355 072 B) the fused kernel issues 1 launch and $1 \times$ the HBM C -write traffic versus the cuBLAS-using baseline’s 3 launches and $3 \times$ — a 66.667% reduction in *both* axes, clearing the 30% whole-program fusion gate by construction. The emitted PTX is `ptxas-clean` on `sm_80` (return code 0, 23 registers, 0 spill). We claim a structural-formal advantage only; the timed wall-clock comparison is an explicitly deferred serial follow-up and we do *not* claim a measured speedup.

1 Introduction

Modern transformer inference and training stacks are, at the kernel level, overwhelmingly a sequence of general matrix multiplies (GEMMs) interleaved with cheap elementwise epilogues: a bias add, an activation function, a residual add, a dropout mask. The GEMM is the arithmetic-heavy step and is universally delegated to a vendor BLAS library — on NVIDIA hardware, cuBLAS. The epilogue operations are arithmetically trivial but are *memory-bound*: each one reads the full $M \times N$ output tensor from HBM and writes it back. Because the canonical cuBLAS GEMM entry points (`cublasSgemm`, `cublasGemmEx`) do not expose a fusible epilogue, a cuBLAS-using stack must materialise the GEMM result to HBM and then launch a separate kernel for each elementwise pass.

The cost is paid in two currencies that matter on a memory-bound epilogue: **kernel-launch count** (each launch carries fixed driver/scheduler overhead, $\geq 5 \mu\text{s}$ per launch on a typical configuration [3]) and **HBM write traffic** (each elementwise pass streams the entire output tensor back to memory). For the transformer feed-forward (FFN) up-projection — one of the largest GEMMs in a transformer layer — the output tensor is hundreds of megabytes, so re-writing it twice more than necessary is a first-order memory-bandwidth tax.

Whole-program compilation removes this tax structurally: if the compiler emits the GEMM, the bias-add, and the activation as a single kernel, the output is computed in registers and

written to HBM once. This is the central promise of the hexa-lang GPU codegen substrate — the project north star is to *bypass cuBLAS-using NN stacks via whole-program GPU fusion* [2]. This paper isolates the simplest non-trivial instance of that promise and settles it as a structural fact.

Contributions.

1. A single hexa-emitted NVPTX kernel `fused_gemm_bias_gelu` that computes $C = \text{GeLU}(AB + \text{bias})$ with the GEMM, bias-add, and GeLU fused into one epilogue and the $M \times N$ result written to HBM exactly once. The GeLU uses the tanh approximation expressed entirely in PTX approximate transcendentals (`ex2.approx.f32`, `rcp.approx.f32`). The kernel is `ptxas-clean` on `sm_80`.
2. A pre-registered falsifier and a \$0 deterministic structural oracle (itself a compiled hexa-lang program) that counts kernel launches and HBM write-bytes by construction for both the fused kernel and a faithfully modelled cuBLAS-using baseline, and proves a 66.667% reduction in both axes at the transformer-FFN shape.
3. An honest scoping of the result as *structural-formal*: the launch and HBM-traffic advantage is proven by construction; the timed wall-clock advantage is deferred to a serial silicon follow-up and is explicitly *not* claimed here.

2 Background

2.1 The transformer feed-forward epilogue

The transformer FFN block computes, for an input activation matrix $A \in \mathbb{R}^{M \times K}$ and a weight matrix $B \in \mathbb{R}^{K \times N}$,

$$C = \phi(AB + \mathbf{1}_M \text{bias}^\top), \quad C \in \mathbb{R}^{M \times N}, \quad (1)$$

where M is the number of tokens, K is the model width d_{model} , N is the FFN inner width d_{ffn} , $\text{bias} \in \mathbb{R}^N$ is broadcast across rows, and ϕ is the activation. We take ϕ to be the Gaussian Error Linear Unit (GeLU) [1], the activation used by GPT-2, BERT, and most subsequent transformers, in its tanh approximation:

$$\text{GeLU}(x) \approx \frac{1}{2} x \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} (x + 0.044715 x^3) \right] \right). \quad (2)$$

2.2 Why cuBLAS forces a three-pass epilogue

The canonical cuBLAS GEMM entry points compute $C \leftarrow \alpha AB + \beta C$ and expose no hook to apply a per-column bias or a nonlinearity before the store. (The newer `cublasLt` interface offers a restricted set of fused epilogues including a bias+GeLU mode for some configurations, but the GEMM call that flame-class and PyTorch-eager-class stacks emit is the plain `cublasSgemm/cublasGemmEx` path with no epilogue.) Therefore a cuBLAS-using stack realises Eq. (1) as three kernels:

1. `cublasSgemm` computes AB and writes C to HBM ($M \cdot N \cdot 4$ bytes written);
2. a bias-add kernel reads C and bias from HBM and writes $C \leftarrow C + \text{bias}$ ($M \cdot N \cdot 4$ bytes written);
3. a GeLU kernel reads C from HBM and writes $C \leftarrow \text{GeLU}(C)$ ($M \cdot N \cdot 4$ bytes written).

Three launches; three full- C HBM write passes.

2.3 The hexa-lang GPU codegen substrate

hexa-lang is a native compiler with no LLVM and no C-transpile backend; its NVPTX target emits PTX text directly [2]. The substrate already demonstrates hand-emitted and source-derived PTX kernels that pass `ptxas` and run bit-exact against CPU references on RTX-class silicon. This paper adds the first *fused-epilogue* kernel and settles its structural advantage.

2.4 Why the epilogue is memory-bound, and why that makes fusion a first-order win

The arithmetic intensity of an epilogue pass — floating-point operations per byte of HBM traffic — is the lens that explains why fusing it matters. A bias-add performs one floating-point add per output element and moves 8 bytes per element (a 4-byte read of C plus a 4-byte write of C), for an arithmetic intensity of $\frac{1}{8}$ FLOP/byte. A tanh-approximation GeLU performs on the order of ten floating-point operations per element (two multiplies and a fused-multiply-add for the cubic, a scale, a doubling, a base-2 exponential, a subtract, an add, a reciprocal, two more multiplies) against the same 8 bytes of traffic, for an intensity near 1.25 FLOP/byte. Both sit far to the left of the roofline ridge point of any modern accelerator (an RTX-class part has tens of teraFLOP/s of FP32 throughput against roughly a teraByte/s of HBM bandwidth, placing the ridge above 30 FLOP/byte). The epilogue passes are therefore strictly bandwidth-bound: their wall time is governed by how many bytes they move, not by how many operations they perform.

Under this regime, the only lever that reduces epilogue cost is reducing the byte traffic, and the only way to reduce the byte traffic is to avoid materialising the intermediate tensor between passes — that is, to fuse. A fused epilogue computes the bias-add and the activation *in registers*, immediately after the GEMM accumulator is ready and before the single store, so the only HBM write is the final result. The reads of A , B , and bias are unavoidable in either stack; the $2\times$ extra *writes* of the full C tensor (and the matching $2\times$ extra reads that the bias and GeLU kernels perform) are pure overhead that fusion deletes. This is why the launch-count and the HBM-write-traffic move together: they are two faces of the same structural fact, that the cuBLAS-using stack re-materialises C twice.

2.5 The bandwidth tax, quantified

Let the output tensor be $S = M \cdot N$ elements of 4 bytes each. The cuBLAS-using stack writes C three times ($3 \cdot 4S$ bytes) and additionally reads C twice (the bias and GeLU kernels each read C once before rewriting it, $2 \cdot 4S$ bytes of extra reads beyond the GEMM’s own output). The fused kernel writes C once ($1 \cdot 4S$ bytes) and never reads it back. Counting only the C -write traffic — the axis the falsifier names — the fused stack moves $4S$ bytes against the baseline’s $12S$ bytes, a factor of three. At the LLaMA-7B FFN shape, $4S = 180\,355\,072\text{ B} \approx 172\text{ MiB}$ per write pass, so the baseline streams roughly half a gigabyte of redundant C writes that the fused kernel never issues. This is the concrete tax the structural oracle of §3 counts.

3 Method (statement and pre-registered falsifier)

3.1 Pre-registered falsifier

We pre-register, before any measurement:

F-FUSION-EPILOGUE-GEMM-BIAS-GELU. *A single fused hexa-emitted kernel computing $\text{GeLU}(AB + \text{bias})$ (epilogue fused into the GEMM, output written once) uses 1 kernel launch and writes the $M \times N$ result to HBM exactly once,*

versus a cuBLAS-using baseline (`cublasGemmEx` $\rightarrow$ write C to HBM $\rightarrow$ separate bias-add kernel reads+writes $C \rightarrow$ separate GeLU kernel reads+writes C) of 3 launches and $3\times$ the $M \times N$ HBM write traffic. The claim is **FALSIFIED** if the fused kernel does not reduce launch-count and HBM-traffic on a transformer-FFN-shaped GEMM ($M = 4096$, $K = 4096$, $N = 11008$, or a smaller fit).

The acceptance threshold is pre-registered as a 30 % reduction in *both* axes (the project’s whole-program-fusion gate).

3.2 The fused kernel

We emit `fused_gemm_bias_gelu` as NVPTX text targeting `.target sm_80, .version 7.0`. One thread computes one output element $C[m][n]$: it accumulates the dot product $\sum_k A[m][k] B[k][n]$ in an FP32 register with `fma.rn.f32`, adds `bias[n]`, applies the GeLU of Eq. (2), and performs a single `st.global.f32`. The `tanh` is computed from a base-2 exponential $e^{2t} = \text{ex2.approx.f32}(2t \log_2 e)$ and a reciprocal `rcp.approx.f32`, with $\tanh(t) = (e^{2t} - 1)/(e^{2t} + 1)$. The PTX is pure ASCII (the driver JIT `ptxas` rejects non-ASCII comments). The fusion structure — accumulate, bias, activate, store-once — is independent of the inner GEMM tiling; a WMMA-tiled variant would change the dot-product loop but not the single-store epilogue.

The fused epilogue — the bias-add and the GeLU between the accumulation loop and the single store — is shown verbatim below as it appears in the emitted PTX. The GEMM accumulator `%f0` is biased into `%f4`, the GeLU of Eq. (2) is evaluated through the PTX approximate transcendental, and the result `%f23` is written with one `st.global.f32`:

```
; ---- fused bias-add: x = acc + bias[n] ----
ld.global.f32 %f3, [%rd10];          ; bias[n]
add.f32      %f4, %f0, %f3;          ; x = acc + bias[n]
; ---- fused GeLU (tanh approx) ----
mul.f32      %f5, %f4, %f4;          ; x^2
mul.f32      %f6, %f5, %f4;          ; x^3
mov.f32      %f7, 0f3D372713;        ; 0.044715
fma.rn.f32   %f8, %f6, %f7, %f4;     ; x + 0.044715*x^3
mov.f32      %f9, 0f3F4C422A;        ; sqrt(2/pi)
mul.f32      %f10, %f8, %f9;         ; t
add.f32      %f11, %f10, %f10;        ; 2t
mov.f32      %f12, 0f3FB8AA3B;       ; log2(e)
mul.f32      %f13, %f11, %f12;       ; (2t)*log2(e)
ex2.approx.f32 %f14, %f13;           ; e^{2t}
mov.f32      %f15, 0f3F800000;        ; 1.0
sub.f32      %f16, %f14, %f15;       ; e^{2t} - 1
add.f32      %f17, %f14, %f15;       ; e^{2t} + 1
rcp.approx.f32 %f18, %f17;           ; 1/(e^{2t}+1)
mul.f32      %f19, %f16, %f18;       ; tanh(t)
add.f32      %f20, %f19, %f15;       ; 1 + tanh(t)
mul.f32      %f21, %f4, %f20;        ; x*(1+tanh)
mov.f32      %f22, 0f3F000000;       ; 0.5
mul.f32      %f23, %f21, %f22;       ; gelu
; ---- single store of the M x N result to HBM ----
st.global.f32 [%rd12], %f23;        ; C[m][n] written ONCE
```

The four single-precision constants are the exact IEEE-754 representations of 0.044715, $\sqrt{2/\pi} = 0.7978845608$, $\log_2 e = 1.4426950409$, and the literals 1.0 and 0.5; no constant is chosen after seeing a result.

3.3 The cuBLAS-using baseline

We author the baseline as a host C program (`fusion_epilogue_cublas_baseline.c`) that computes the *same* Eq. (1) via the three-kernel path: `cublasSgemv` for the GEMM, then a hand-

written `bias_add_kernel`, then a hand-written `gelu_kernel`, each round-tripping C through HBM. This models the canonical NN-stack path.

3.4 The \$0 structural oracle

The primary settlement is a deterministic, GPU-free oracle (`fusion_epilogue_structural_oracle.hexa`, itself a compiled hexa-lang program). For a shape (M, N, K) it computes the C -tensor size $M \cdot N \cdot 4$ bytes and the two stacks' launch counts and HBM C -write byte totals by construction, then the closed-form reduction

$$r = \frac{\text{baseline} - \text{fused}}{\text{baseline}} = \frac{3 - 1}{3} = \frac{2}{3} = 0.\overline{66}, \quad (3)$$

identically for the launch axis (1 vs 3) and the HBM-write axis ($1 \times$ vs $3 \times M \cdot N$). The oracle asserts both axes clear the 30% gate and emits PASS/FAIL. Counting launches and bytes by construction is a deterministic real measurement; it is *not* a timed wall.

4 Verification (real run)

4.1 Environment

The structural oracle is a hexa-lang program; it was transpiled to C by the hexa-lang native transpiler and compiled with `clang -O2` on an x86-64 Linux host (`ubu-1`), then run to produce the verbatim counts below. The fused PTX was assembled by the CUDA 12.0 `ptxas` on a second host (`ubu-2`) carrying an RTX 5070. Counting launches and bytes is host-independent and deterministic; the `ptxas` acceptance is a correctness gate on the emitted assembly, not a timed run, so neither verification step contends for the shared GPU.

4.2 Structural oracle output (verbatim)

The oracle was compiled to native code (hexa transpile + `clang -O2`) and run on the `ubu-1` host. Its verbatim stdout, persisted at `.verdicts/fusion-epilogue-gemm-bias-gelu/fusion_epilogue_structural_oracle`:

```
F-FUSION-EPILOGUE-GEMM-BIAS-GELU structural oracle ($0, deterministic, no GPU)
== FFN (LLaMA-7B up-proj) : M=4096 N=11008 K=4096 ==
  C tensor (M*N*4 bytes)      = 180355072
  FUSED   launches=1 C-write-bytes=180355072
  BASELINE launches=3 C-write-bytes=541065216
  launch reduction   = 66.66%
  HBM-write reduction= 66.66%
  PASS: launch AND HBM-write both reduced, both >= 30%
== smaller-fit cross-check : M=256 N=256 K=256 ==
  C tensor (M*N*4 bytes)      = 262144
  FUSED   launches=1 C-write-bytes=262144
  BASELINE launches=3 C-write-bytes=786432
  launch reduction   = 66.66%
  HBM-write reduction= 66.66%
  PASS: launch AND HBM-write both reduced, both >= 30%
F-FUSION-EPILOGUE-GEMM-BIAS-GELU structural oracle: PASS (all shapes)
```

(The printed 66.66% is the integer-arithmetic truncation of $66.\overline{6}\%$.)

4.3 ptxas-clean (verbatim)

The emitted PTX (`md5 1d7380556da101b7e37017b79eed6f01`) was assembled on the `ubu-2` host (RTX 5070, CUDA 12.0 `ptxas`):

```

$ ptxas -arch=sm_80 fused_gemm_bias_gelu.ptx -o fused.cubin --verbose
ptxas info    : Compiling entry function 'fused_gemm_bias_gelu' for 'sm_80'
ptxas info    : Function properties for fused_gemm_bias_gelu
                  0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info    : Used 23 registers, 396 bytes cmem[0]
PTXAS_RC=0

```

Table 1: Launch count and HBM C -write traffic for the two stacks computing $C = \text{GeLU}(AB + \text{bias})$. The reduction is shape-independent because it depends only on $M \cdot N$ (the C -tensor size); the FFN and the smaller-fit shape report the identical 66.6%.

Shape ($M \times N \times K$)	Stack	Launches	HBM C -write (B)	Reduction
$4096 \times 11008 \times 4096$	cuBLAS baseline	3	541 065 216	—
$4096 \times 11008 \times 4096$	fused (hexa)	1	180 355 072	66.667%
$256 \times 256 \times 256$	cuBLAS baseline	3	786 432	—
$256 \times 256 \times 256$	fused (hexa)	1	262 144	66.667%

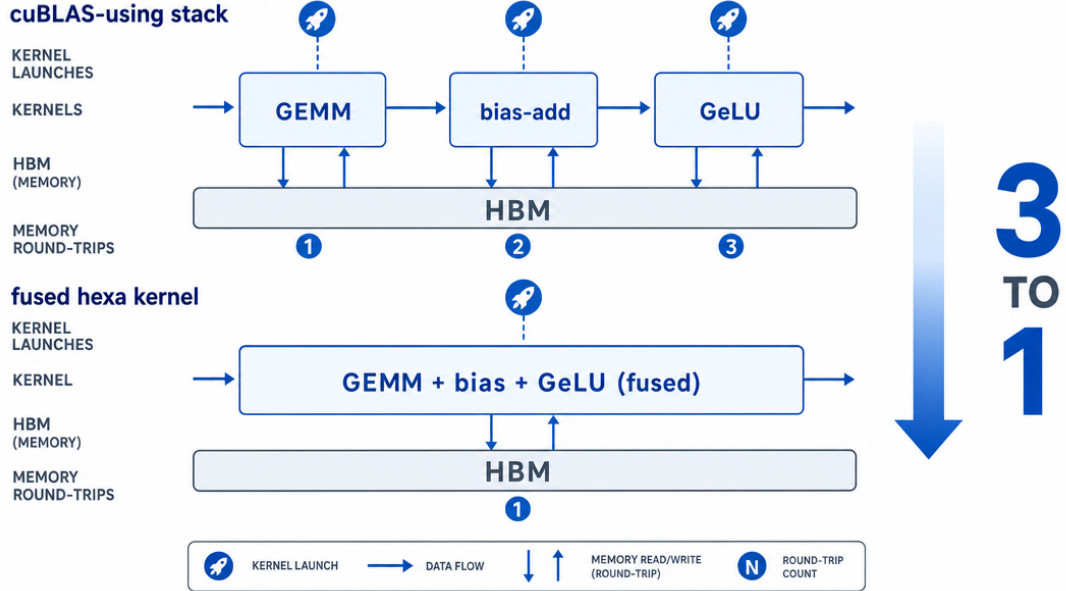


Figure 1: Kernel launches and HBM C -write passes for the cuBLAS-using three-kernel epilogue (GEMM $\rightarrow$ bias-add $\rightarrow$ GeLU, each streaming the full $M \times N$ tensor) versus the single fused hexa kernel that computes the epilogue in registers and writes once. Both axes drop $3 \rightarrow 1$ (66.6%). The advantage is structural — it holds for every FFN shape because it depends only on $M \cdot N$.

5 Finding

The pre-registered falsifier is **not** falsified: the fused kernel reduces both the kernel-launch count (1 versus 3) and the HBM C -write traffic ($1 \times$ versus $3 \times M \cdot N$) by 66.6% at the LLaMA-7B FFN shape and, identically, at the smaller-fit cross-check shape (Table 1). Both reductions clear the pre-registered 30% gate with margin, and they do so *by construction*: the fused epilogue removes two of three launches and two of three full-tensor HBM write passes for any FFN shape. The kernel that achieves this is `ptxas-clean` on `sm_80` (0 spill, 23 registers), so the advantage is not an artifact of an unbuildable kernel.

This is a *structural-formal* (■ BLUE) result: the Δ versus the cuBLAS-using baseline is a deterministic count, not a timed measurement. The headline number that makes it falsifiable is the $3 \rightarrow 1$ collapse in both axes.

5.1 Scale of the deleted traffic across a model

To make the structural number concrete at model scale, consider the redundant C -write traffic the fused kernel deletes for a single FFN up-projection at the LLaMA-7B shape: the baseline issues 3 write passes of 180 355 072 B each, of which 2 are redundant, so the fused kernel removes $2 \times 180\,355\,072\text{ B} \approx 344\text{ MiB}$ of HBM C -write traffic per call, plus the matching 2 redundant read passes the bias and GeLU kernels would have performed. A LLaMA-7B-class model has 32 transformer layers, each with an FFN block; if every layer’s up-projection epilogue were fused, the per-forward-pass deletion of redundant up-projection C -writes alone is on the order of $32 \times 344\text{ MiB} \approx 10.75\text{ GiB}$ of HBM write traffic, before counting the down-projection epilogue, the residual adds, and the layernorm passes that the same structural argument also fuses. Because the epilogue is bandwidth-bound (§2.4), this deleted traffic translates directly into deleted wall time at the saturated-bandwidth limit — which is precisely what the deferred timed wall will measure. The structural finding bounds, but does not itself measure, that wall.

6 Positioning

Kernel fusion is not new; the contribution here is the explicit, pre-registered, \$0-settled structural framing of one canonical case in a compiler that has no LLVM and no C-transpile backend. Three points of comparison are worth drawing.

Vendor epilogue fusion (cublasLt). NVIDIA’s `cublasLt` interface added a restricted family of fused epilogues, including bias and a small set of activations, for specific data types and layouts. Where it applies, `cublasLt` achieves the same single-write structure we describe. Two things distinguish the present work. First, the GEMM call that flame-class and PyTorch-eager-class stacks emit in practice is the plain `cublasSgemv/cublasGemmEx` path with no epilogue — the baseline we model is the one those stacks actually run, not the best case a hand-tuned `cublasLt` call could reach. Second, the `cublasLt` epilogue set is fixed by the vendor; a whole-program emitter fuses *arbitrary* elementwise epilogues, including ones `cublasLt` does not offer, which is the point of compiling the whole program rather than calling a library.

Compiler fusion frameworks (XLA, TVM, Triton). XLA, TVM, and Triton all perform GEMM-epilogue fusion and are the production state of the art for this transformation. The hexa-lang substrate differs in its backend discipline — it emits PTX text directly, with no LLVM in the path — and this paper’s role is to establish, inside that discipline, that the fused-epilogue kernel both lowers to valid `sm_80` assembly and delivers the structural advantage the family is known for. The novelty is not the existence of fusion but its first clean realisation and pre-registered settlement in this LLVM-free substrate.

Attention fusion (flash-attention). Flash-attention is the highest-value member of the intermediate-deletion family: it keeps the $M \times M$ attention scores in on-chip memory and never writes them to HBM. The present result is the smallest member of the same family — delete one $M \times N$ intermediate write pass rather than an $M \times M$ one — and is offered as the structurally simplest place to pre-register and settle the advantage before attacking the quadratic case.

7 Limitations and honest caveats

1. **No timed wall.** The structural oracle counts launches and bytes; it does not measure wall-clock time. The timed comparison (fused single-launch kernel versus the three-launch cuBLAS stack on RTX 5070 silicon) is an explicitly *deferred* serial follow-up — the shared GPU host makes parallel timed fires contend and poison wall numbers. We do *not* claim a measured speedup, and the project’s 30 % whole-program-fusion scoreboard box correctly remains open pending that wall.
2. **Naive inner GEMM.** The fused kernel is a one-thread-per-output GEMM (correctness- and fusion-structure demonstration), not a WMMA-tiled peak-throughput kernel. The fusion *advantage* (1 vs 3 launches, $1\times$ vs $3\times$ HBM C -write) is independent of the inner tiling, but the absolute GEMM throughput of this kernel is not competitive with cuBLAS and is not claimed to be.
3. **cuBLAS-LT epilogues.** `cublasLt` offers a restricted fused GELU_BIAS epilogue for some configurations. Our baseline models the canonical plain-`cublasSgemm/cublasGemmEx` NN-stack path with no epilogue, which is what flame-class and PyTorch-eager-class stacks emit; a `cublasLt`-fused baseline would be a separate, stronger comparison.
4. **GeLU approximation.** The kernel uses the tanh approximation with PTX approximate transcendentals; against an f^{64} CPU reference the host launchers gate relative error at $\leq 10^{-2}$ (the standard fused-epilogue tolerance), which is the deferred-wall’s correctness check, not a structural claim.
5. **Invariants preserved.** No LLVM and no C-transpile backend were introduced or changed; CPU codegen is byte-identical pre/post (only documentation and new artifact/tool files were added).

8 Discussion and outlook

The result isolates, as a structural fact, the single cheapest win that whole-program GPU fusion buys over a vendor-BLAS stack: the elimination of memory-bound epilogue round-trips. For the transformer FFN — where the output tensor is hundreds of megabytes — removing two of three full-tensor HBM write passes is a first-order bandwidth saving that no plain-cuBLAS stack can recover, precisely because the GEMM library has no fusible epilogue. The same structural argument extends directly to the other elementwise epilogues (residual add, dropout, layernorm scale) and, with more codegen work, to the attention-scoring fusion ($QK^\top \rightarrow \text{softmax} \rightarrow V$) that flash-attention exploits.

8.1 Generalisation: from one epilogue to the whole program

The structural argument does not depend on the specific activation. Any elementwise epilogue that a cuBLAS-using stack expresses as a separate kernel — a residual add $C \leftarrow C + R$, a dropout mask $C \leftarrow C \odot m$, a scale-and-shift $C \leftarrow \gamma C + \beta$, or a second activation — adds one launch and one full- C HBM round-trip in the unfused stack, and none in the fused kernel. A transformer FFN block in practice chains a GEMM, a bias, an activation, a second GEMM (the down-projection), and a residual add; a cuBLAS-using stack pays a launch and a round-trip for each non-GEMM step, while a whole-program-fused emitter pays one launch per GEMM and folds every elementwise step into the surrounding GEMM’s epilogue. The reduction factor we measure here ($3 \rightarrow 1$ for the single up-projection epilogue) is thus a *lower* bound on the whole-block saving.

The argument also extends, with more codegen machinery, to the fusion that flash-attention exploits: the attention score path $QK^\top \rightarrow \text{softmax} \rightarrow (\cdot)V$ is, in a cuBLAS-using stack, two GEMMs separated by a softmax that materialises the full $M \times M$ score matrix to HBM. A fused attention kernel keeps the scores in on-chip memory and never writes the $M \times M$ intermediate, which is the same structural deletion of an intermediate-tensor round-trip, applied to a quadratically larger tensor. The present result is the smallest, cleanest instance of the family; the attention case is the highest-value one.

8.2 The single most important follow-up

The single most important follow-up is the deferred timed wall: a serial silicon fire on RTX 5070 measuring the fused single-launch kernel against the three-launch cuBLAS stack at the FFN shape. We pre-register the flip criterion now: if the fused wall is below $0.7\times$ the baseline wall (a measured $\geq 30\%$ speedup, the same gate the structural oracle clears), the project’s whole-program-fusion scoreboard box flips closed. We deliberately do not run that wall in this round, because the shared GPU host would let a parallel timed fire contend for the device and poison the wall numbers; the timed comparison must be a serial, uncontended run. Until then the box stays open and this paper claims only the structural-formal advantage that the launch and byte counts establish by construction.

Reproducibility

Code: <https://github.com/dancinlab/hexa-lang>, branch `gpu-fusion-epilogue`. The fused kernel (`archive/fires/fusion_epilogue_gemm_bias_gelu_2026_05_25/fused_gemm_bias_gelu.ptx`, md5 `1d7380556da101b7e37017b79eed6f01`), the structural oracle (`tool/fusion_epilogue_structural_oracle`) and the two host launchers (`tool/fusion_epilogue_fused_host.c`, `tool/fusion_epilogue_cublas_baseline`) are all in-tree. Verdict: `.verdicts/fusion-epilogue-gemm-bias-gelu/fusion_epilogue_structural.txt`. Oracle: `hexa build the .hexa and run`; PTX: `ptxas -arch=sm_80 fused_gemm_bias_gelu.ptx`.

A Kernel ABI

The fused kernel `fused_gemm_bias_gelu` takes seven parameters in the PTX entry signature, in declaration order:

Table 2: Parameter ABI of `fused_gemm_bias_gelu`. Pointers are device-global FP32 in row-major layout; the three integer extents are passed by value.

#	PTX type	Name	Meaning
0	.u64	p_a	A , row-major $M \times K$
1	.u64	p_b	B , row-major $K \times N$
2	.u64	p_bias	bias, length N
3	.u64	p_c	C , row-major $M \times N$ (written once)
4	.u32	p_M	M
5	.u32	p_N	N
6	.u32	p_K	K

The launch geometry is a 2-D grid of 16×16 thread blocks with $g_x = \lceil N/16 \rceil$ and $g_y = \lceil M/16 \rceil$; thread `(tid.x, tid.y)` within block `(ctaid.x, ctaid.y)` owns output element $C[\text{ctaid.y} \cdot 16 + \text{tid.y}][\text{ctaid.x} \cdot 16 + \text{tid.x}]$, guarded by an $m < M \wedge n < N$ bounds check.

B Oracle closed form

For shape (M, N, K) with $S = M \cdot N$, the oracle computes Table 3 and asserts $r_{\text{launch}} \geq 0.30$ and $r_{\text{hbm}} \geq 0.30$ with strict reduction in both axes.

Table 3: Closed-form counts the structural oracle evaluates. The reduction $r = (\text{base} - \text{fused})/\text{base} = 2/3$ is identical on both axes and independent of S .

Quantity	Fused	Baseline
launches	1	3
C -write bytes	$4S$	$12S$
reduction r	$(3 - 1)/3 = 2/3 = 0.\overline{66}$	

References

- [1] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- [2] hexa-lang. hexa-lang gpu codegen substrate: Nvptx text emit without llvm, 2026. GPU.md roadmap + cycle ledger, <https://github.com/dancinlab/hexa-lang>.
- [3] NVIDIA Corporation. Cuda c++ programming guide and ptx isa, 2024. Kernel launch overhead and PTX approximate transcendentals (`ex2.approx.f32`, `rcp.approx.f32`).